

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 08 · LECTURE NOTES

Distributed Data

Splitting it up, and putting it back together.

MODULE OUTCOMES

Compare partitioning schemes and their rebalancing behaviour.

Explain consistent hashing and virtual nodes.

Explain two-phase commit and why it blocks.

Explain sagas and compensating actions.

Explain CRDTs and when they genuinely apply.

1 Partitioning

Scheme	How it works	The problem with it
Range partitioning	Each node owns a contiguous range of the key space.	Hot spots. Sequential keys — timestamps, auto-increment IDs — all land on one node, so the newest and busiest data is never spread.
Hash partitioning	$\text{node} = \text{hash}(\text{key}) \bmod n$.	Changing n remaps almost everything. Going from 10 nodes to 11 moves roughly 10/11 of all keys, not 1/11 — which makes adding capacity a full data migration.
Consistent hashing	Hash keys and nodes onto a ring; a key belongs to the first node clockwise.	Only about $1/n$ of keys move when a node is added. Requires virtual nodes to balance — see below.
Directory	An explicit lookup table mapping key ranges to nodes.	Maximum flexibility — you can move anything anywhere — and the directory is a single point of failure that must itself be replicated (Module 06).
By tenant	All of one customer's data on one node.	Simple, and transactions stay local , which is a large advantage (§4). One enormous tenant breaks it , and there is always one.

CONSISTENT HASHING, AND WHY VIRTUAL NODES ARE MANDATORY

Hash both keys and node identifiers onto a ring, and assign each key to the first node encountered clockwise from its position.

Adding a node takes keys only from its immediate successor on the ring, so roughly $1/n$ of the data moves rather than nearly all of it. That is the entire point, and it is what makes elastic scaling possible at all.

But with one ring position per node the load is badly uneven. Random placement of n points on a circle produces arcs of very different sizes — the largest is typically several times the average — so some nodes get far more data than others.

So each physical node is given 100 to 200 virtual positions on the ring. The variance averages out, load becomes close to uniform, and **removing a node spreads its keys across many successors rather than dumping them all on one** — which matters enormously during a failure, since dumping one node's entire load onto its neighbour is how a single failure cascades. **Without virtual nodes, consistent hashing does not work in practice**, and this is frequently omitted from descriptions of it.

2 Transactions across partitions

```

PHASE 1 -- PREPARE
  coordinator -> participants: "can you commit?"
  each does the work, makes it DURABLE, replies YES or NO.
  A YES is a PROMISE that cannot be retracted.

PHASE 2 -- COMMIT or ABORT
  all YES -> "commit"; any NO -> "abort"

THE PROBLEM
  a participant that voted YES and then hears nothing is STUCK:
    it cannot commit -- someone may have voted NO
    it cannot abort -- it promised
    it cannot ask the others -- they may not know either
  so it HOLDS ITS LOCKS AND WAITS.

```

The coordinator is a single point of failure that can block the system indefinitely -- Module 01's ambiguity, with locks held.

3PC removes blocking under a synchrony assumption real networks do not satisfy. Modern systems instead make the COORDINATOR fault-tolerant by running it on consensus (Module 06).

Two-phase commit is an atomicity protocol, not a fault-tolerance protocol — it ensures all participants reach the same decision, and says nothing about what happens when the decider fails. **Those are different problems**, and recognising that 2PC only solves the first explains both why it blocks and why the modern answer is to compose it with consensus rather than to replace it.

SAGAS: GIVE UP ATOMICITY, KEEP THE OUTCOME

Split the distributed transaction into a sequence of local transactions, each paired with a defined *compensating action* that semantically undoes it.

Execute them in order; on failure, run the compensations backwards. Book the flight, then the hotel, then the car; if the car fails, cancel the hotel and cancel the flight.

There is no isolation. Intermediate states are visible to everyone — another observer can see the flight booked while the hotel is still pending, and can act on that. **The application must be designed to tolerate its own intermediate states being seen**, which is a real constraint and the main reason sagas are harder than they look.

And compensation is not rollback. You cannot un-send an email; you send a correction. You cannot un-charge a card; you issue a refund, which leaves two entries on the statement. **The business decides what compensation means** — which makes sagas a domain modelling exercise rather than an infrastructure one, and is why they cannot be provided by a library.

3 CRDTs

CONVERGENCE WITHOUT COORDINATION, BY CONSTRUCTION

If the merge operation is commutative, associative, and idempotent, then replicas converge to the same value regardless of the order in which updates arrive, how many times they arrive, or whether they arrive more than once.

So **no coordination is required at all** — no consensus, no locks, no conflict resolution rule to argue about, and **no silently lost writes** (Module 03 §4). Updates can be applied in any order by any replica and the result is the same.

The standard examples: a grow-only counter (each replica counts its own increments; merge takes the maximum per replica and sums); a grow-only set (merge is union); a two-phase set supporting removal via tombstones; a last-writer-wins register with a deterministic tiebreak; and sequence CRDTs for collaborative text.

The cost is that not every datatype has one, and the metadata can grow without bound. **Deletions require tombstones** — a record that the element was removed — kept so that a late-arriving add does not resurrect it, and garbage-collecting tombstones safely requires knowing every replica has seen them, which is itself a coordination problem.

Case	Does a CRDT work?	Why
Collaborative text editing	Yes.	Sequence CRDTs — RGA, Logoot, Yjs, Automerge. The flagship application , and the one that drove most of the research.
Presence, tag sets, counters	Yes.	The merge semantics are natural: union, maximum, sum.
Shopping cart	Mostly.	Dynamo's original example. Merging by union never loses an add, and a deleted item can reappear if the delete and a concurrent add cross — which Amazon judged preferable to losing an add.
Bank balance	No.	'The balance must never go below zero' is not a mergeable property. Two replicas can each approve a withdrawal that is individually valid and jointly is not.
Unique username	No.	Uniqueness is a global invariant; convergence cannot establish it after the fact.
Game world state	No.	A game needs a referee, not convergence — who shot whom first is a decision, not a merge (Module 09).

The clean test: CRDTs handle invariants that are local, not global. Anything requiring 'exactly one of these exists' or 'this quantity never goes below zero' is a global invariant and needs coordination, because it is a statement about all replicas jointly that no replica can check alone.

4 Choosing

- 1 **Keep it on one node.** A partition boundary you do not have costs nothing to maintain, and single-node transactions are a solved problem (CSCE 608). This is Module 01 §4's argument applied at the data layer.
- 2 **Partition so that transactions stay within one partition.** Co-locate by tenant, by user, by order, by game session — whatever unit the transactions naturally fall within. **This is where the leverage is**, and most cross-partition transactions turn out to be an artifact of the partitioning choice rather than a property of the domain.
- 3 **Use a CRDT**, if the invariant is local (§3).
- 4 **Use a saga** with explicitly defined compensating actions, if the business can say what undoing each step means (§2).
- 5 **Use two-phase commit over a consensus-backed coordinator.** Correct, slow, and a **genuinely available option** — Spanner does essentially this, and dismissing distributed transactions as impossible is out of date.
- 6 **Reconsider the requirement. It is frequently negotiable and is rarely asked about** — 'must these two things be atomic, or must they merely both eventually happen?' has a different answer surprisingly often, and the second is far cheaper.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Kleppmann — Designing Data-Intensive Applications, chapters 6, 7 and 9	Partitioning, transactions, and 2PC, with the blocking problem of §2 explained properly.
Karger et al. — Consistent Hashing and Random Trees (free)	The §1 algorithm, from the paper that introduced it and founded Akamai.
Shapiro, Preguiça, Baquero & Zawirski — A Comprehensive Study of Convergent and Commutative Replicated Data Types (free)	The CRDT reference. Comprehensive, and the taxonomy in the first sections is the useful part.
Garcia-Molina & Salem — Sagas (free)	The §2 alternative, from 1987 — long before microservices rediscovered it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Measure how many keys move** under $\text{hash}(\text{key}) \bmod n$ when n goes from 10 to 11. Compare against consistent hashing.

- 2 Implement consistent hashing with one point per node and **plot the load distribution**.
- 3 **Add 150 virtual nodes each** and plot it again. Report the variance in both cases.
- 4 Remove a node from both and compare where its load goes.
- 5 Implement two-phase commit.
- 6 **Kill the coordinator after the prepare phase** and show a participant stuck holding locks.
- 7 Run the coordinator on your Raft cluster from Module 06 and show the blocking is gone.
- 8 **Implement a saga** with compensations for a three-step booking, and demonstrate a compensating rollback.
- 9 Implement a grow-only counter and a two-phase set as CRDTs. Verify convergence under reordering and duplication.
- 10 **Demonstrate the resurrected-item problem** in a cart CRDT and explain why Amazon accepted it.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compare five partitioning schemes and their failure modes.
- 2 Why does $\text{hash mod } n$ remap almost everything?
- 3 Why are virtual nodes mandatory for consistent hashing?
- 4 Describe 2PC and explain precisely why a participant blocks.
- 5 Why is 2PC not a fault-tolerance protocol?
- 6 What do sagas give up, and why is compensation not rollback?
- 7 What three properties must a CRDT merge have?
- 8 Give the test for whether a CRDT applies.
- 9 Give the six-step order for choosing, and say where the leverage is.